

Trabajo Práctico 7

Conexión entre aplicaciones y MongoDB

Materia: Base de Datos II

Carrera: Tecnicatura Universitaria en Programación a Distancia

Nombre: Facundo Arrieta

Introducción

En este trabajo práctico se desarrolla la conexión entre una aplicación y MongoDB utilizando dos enfoques diferentes: el driver nativo de MongoDB y Mongoose como ODM.

El driver nativo permite una comunicación más directa con el motor de base de datos, ofreciendo mayor control sobre las consultas y operaciones. Por otro lado, Mongoose agrega una capa de abstracción que permite trabajar con esquemas, modelos y validaciones dentro del código de la aplicación.

El objetivo del trabajo es implementar ambos enfoques en Node.js, conectarse a una base de datos llamada laboratorio y realizar inserciones de prueba para comprobar el funcionamiento de la conexión.

Parte 1: Implementación con Driver Nativo

Objetivo de la Parte 1

El objetivo de esta parte es utilizar el driver oficial de MongoDB para conectar una aplicación Node.js con la base de datos laboratorio y cargar tres componentes o sensores de robótica utilizando el método `insertMany()`.

Instalación del driver nativo

Para comenzar, se inicializó el proyecto de Node.js y se instaló el paquete oficial de MongoDB mediante npm.

Comandos utilizados:

```
npm init -y
```

```
PS C:\Users\facun\OneDrive\Escritorio\UTN\2 año - 1º Cuatrimestre\Base de datos 2\TP7_MONGODB> npm init -y
Wrote to C:\Users\facun\OneDrive\Escritorio\UTN\2 año - 1º Cuatrimestre\Base de datos 2\TP7_MONGODB\package.json:

{
  "name": "tp7_mongodb",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}

PS C:\Users\facun\OneDrive\Escritorio\UTN\2 año - 1º Cuatrimestre\Base de datos 2\TP7_MONGODB>
```

npm pkg set type=module

```
npm pkg set type=module
```

npm install mongodb

```
PS C:\Users\facun\OneDrive\Escritorio\UTN\2 año - 1º Cuatrimestre\Base de datos 2\TP7_MONGODB> npm install mongodb
added 12 packages, and audited 13 packages in 4s
found 0 vulnerabilities
```

Archivo creado

Se creó el archivo:

driver_lab.js

Este archivo contiene el código necesario para conectarse a MongoDB utilizando el driver nativo.

Código utilizado en driver_lab.js

```
import { MongoClient } from 'mongodb';
```

```
const url = 'mongodb://localhost:27017/';
```

```
const client = new MongoClient(url);
```

```
const dbName = 'laboratorio';
```

```
async function main() {
```

```
  await client.connect();
```

```
  console.log('Conexión exitosa a MongoDB');
```

```
const db = client.db(dbName);

const collection = db.collection('componentes');


const sensores = [

  {
    nombre: 'Sensor de Humedad',
    tipo: 'Sensor',
    stock: 10
  },
  {
    nombre: 'Servo Motor',
    tipo: 'Actuador',
    stock: 5
  },
  {
    nombre: 'LED RGB',
    tipo: 'Componente electrónico',
    stock: 20
  }
];

const resultado = await collection.insertMany(sensores);

console.log('Sensores insertados correctamente');
console.log(resultado.insertedIds);
}

try {
  await main();
}
```

```
} catch (error) {  
    console.error('Error:', error);  
}  
} finally {  
    await client.close();  
    console.log('Conexión cerrada');  
}
```

Explicación del código

En primer lugar, se importa MongoClient desde el paquete mongodb. Luego se define el string de conexión local:

```
const url = 'mongodb://localhost:27017/';
```

Este string indica que la aplicación se conectará a una instancia local de MongoDB, utilizando el puerto 27017.

Después se crea una instancia del cliente:

```
const client = new MongoClient(url);
```

A continuación, se define el nombre de la base de datos:

```
const dbName = 'laboratorio';
```

Dentro de la función main(), se establece la conexión con MongoDB, se selecciona la base de datos laboratorio y se accede a la colección componentes.

Luego se crea un array con tres componentes de robótica:

- Sensor de Humedad
- Servo Motor
- LED RGB

Finalmente, se utiliza el método insertMany() para insertar los tres documentos en la colección.

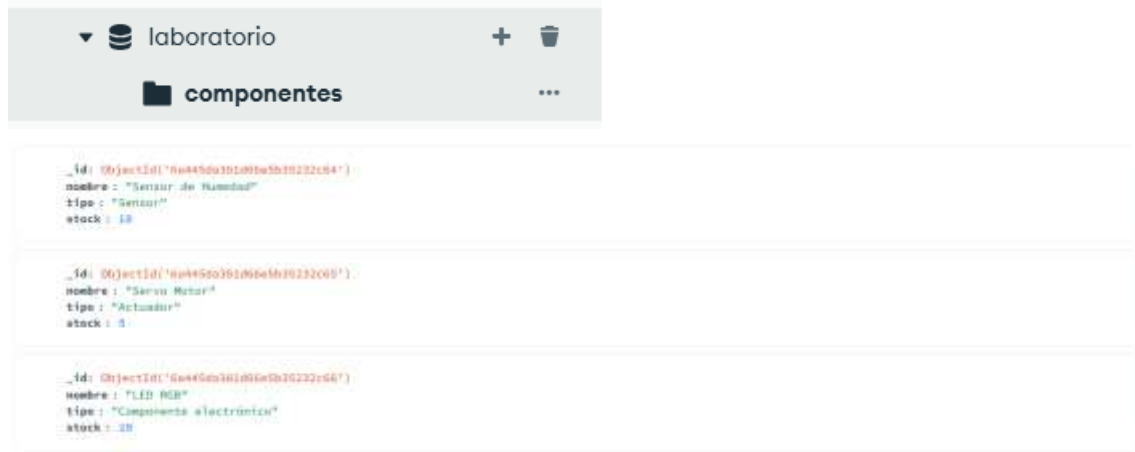
Ejecución del archivo

Para ejecutar el archivo se utilizó el siguiente comando:

```
node driver_lab.js
```

```
PS C:\Users\facun\OneDrive\Escritorio\UTN\2 año - 1º Cuatrimestre\Base de datos 2\TP7_MONGODB> node driver_lab.js
Conexión exitosa a MongoDB
Sensores insertados correctamente
{
  '0': new ObjectId('6a445da381d66e5b35232c64'),
  '1': new ObjectId('6a445da381d66e5b35232c65'),
  '2': new ObjectId('6a445da381d66e5b35232c66')
}
Conexión cerrada
```

Verificación en MongoDB



Resultado obtenido

La ejecución del archivo permitió conectarse correctamente a MongoDB e insertar tres documentos en la colección componentes de la base de datos laboratorio.

Con esto se comprueba que la aplicación Node.js pudo comunicarse con MongoDB mediante el driver nativo.

Parte 2: Implementación con Mongoose

Objetivo de la Parte 2

El objetivo de esta parte es utilizar Mongoose como ODM para conectar la aplicación con MongoDB, definir un esquema para los componentes y realizar una inserción de prueba.

Mongoose permite trabajar con modelos y esquemas, lo que ayuda a mantener una estructura más ordenada en la aplicación.

Instalación de Mongoose

Para utilizar Mongoose, se instaló el paquete correspondiente mediante npm.

Comando utilizado:

```
npm install mongoose
```

```
Conexión cerrada
PS C:\Users\facun\OneDrive\Escritorio\UTN\2 año - 1º Cuatrimestre\Base de datos 2\TP7_MONGODB> npm install mongoose
added 8 packages, and audited 21 packages in 6s

1 package is looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

Archivo creado

Se creó el archivo:

mongoose_lab.js

Este archivo contiene la conexión a MongoDB mediante Mongoose, la definición del esquema, la creación del modelo y una inserción de prueba.

Código utilizado en mongoose_lab.js

```
import mongoose from 'mongoose';
```

```
const url = 'mongodb://localhost:27017/laboratorio';
```

```
const componenteSchema = new mongoose.Schema(
{
  nombre: {
    type: String,
    required: true
  },
  stock: {
    type: Number
  }
},
{
  timestamps: true
});
```

```
const Componente = mongoose.model('Componente', componenteSchema);
```

```
async function main() {
```

```
  await mongoose.connect(url);
```

```
  console.log('Conexión exitosa a MongoDB con Mongoose');
```

```
  const componente = await Componente.create({
```

```
    nombre: 'Sensor Ultrasónico',
```

```
    stock: 12
```

```
  });
```

```
  console.log('Componente insertado correctamente:');
```

```
  console.log(componente);
```

```
  return 'Proceso finalizado';
```

```
}
```

```
try{
```

```
  const resultado = await main();
```

```
  console.log(resultado);
```

```
} catch (error) {
```

```
  console.error('Error en la conexión o ejecución:', error);
```

```
} finally {
```

```
  await mongoose.connection.close();
```

```
  console.log('Conexión cerrada');
```

```
}
```

Explicación del código

En primer lugar, se importa Mongoose:

```
import mongoose from 'mongoose';
```

Luego se define el string de conexión a la base de datos laboratorio:

```
const url = 'mongodb://localhost:27017/laboratorio';
```

Después se define un esquema llamado componenteSchema. Este esquema indica que cada componente tendrá un campo nombre de tipo String y obligatorio, y un campo stock de tipo Number.

También se activa la opción:

```
timestamps: true
```

Esta opción permite que Mongoose genere automáticamente los campos createdAt y updatedAt.

Luego se crea el modelo:

```
const Componente = mongoose.model('Componente', componenteSchema);
```

El modelo representa la colección con la que se va a trabajar desde el código.

Finalmente, se realiza una inserción de prueba utilizando:

```
Componente.create()
```

Ejecución del archivo

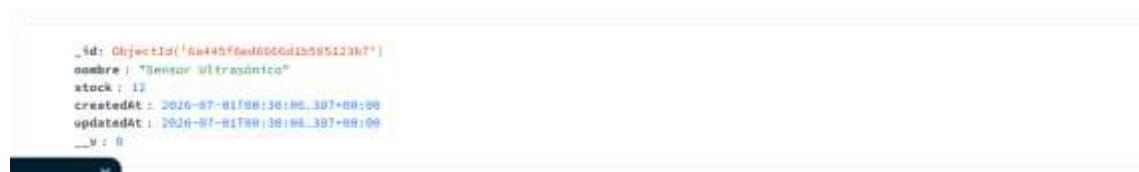
Para ejecutar el archivo se utilizó el siguiente comando:

```
node mongoose_lab.js
```



```
PS C:\Users\facun\OneDrive\Escritorio\UTN\2 año - 1º Cuatrimestre\Base de Datos 2\1º MongoDB> node mongoose_lab.js
Conexión exitosa a MongoDB con Mongoose
Componente insertado correctamente:
{
  nombre: 'Sensor Ultrasónico',
  stock: 12,
  _id: new ObjectId('6a445f8ed6666d1b505123b7'),
  createdAt: 2026-07-01T00:30:06.387Z,
  updatedAt: 2026-07-01T00:30:06.387Z,
  __v: 0
}
Proceso finalizado
Conexión cerrada
```

Verificación en MongoDB



```
{
  "_id": "ObjectId('6a445f8ed6666d1b505123b7')",
  "nombre": "Sensor Ultrasónico",
  "stock": 12,
  "createdAt": "2026-07-01T00:30:06.387+00:00",
  "updatedAt": "2026-07-01T00:30:06.387+00:00",
  "__v": 0
}
```

Resultado obtenido

La ejecución del archivo permitió conectarse correctamente a MongoDB utilizando Mongoose. Además, se pudo insertar un componente de prueba respetando el esquema definido en el código.

También se generaron automáticamente los campos de fecha createdAt y updatedAt, gracias a la opción timestamps: true.

Parte 3: Reflexión Final

1. ¿Cuál es la función del driver como “traductor” entre la aplicación y MongoDB?

La función del driver es actuar como intermediario entre la aplicación y MongoDB. El driver permite que el código escrito en un lenguaje de programación, como JavaScript en Node.js, pueda comunicarse con la base de datos.

Además, se encarga de gestionar la conexión, enviar las operaciones al servidor de MongoDB y recibir las respuestas. De esta manera, la aplicación puede ejecutar operaciones como insertar, consultar, actualizar o eliminar documentos sin comunicarse directamente con el motor de base de datos.

2. ¿A qué formato convierte el driver las estructuras de datos nativas para enviarlas al servidor?

El driver convierte las estructuras de datos nativas del lenguaje, como los objetos de JavaScript, al formato BSON.

BSON es el formato que utiliza MongoDB para representar y transferir documentos. Por ejemplo, un objeto de JavaScript puede ser convertido por el driver para que MongoDB pueda almacenarlo correctamente como documento dentro de una colección.

3. ¿En qué situación de un proyecto de ingeniería elegirían usar el driver nativo en lugar de Mongoose?

Elegiría usar el driver nativo en proyectos simples, scripts pequeños, microservicios o procesos donde el rendimiento sea muy importante.

El driver nativo es conveniente cuando se necesita una comunicación más directa con MongoDB, sin capas adicionales de abstracción. También es útil cuando se requiere mayor control sobre las consultas o cuando el modelo de datos es flexible y puede cambiar con frecuencia.

En cambio, Mongoose sería más recomendable en aplicaciones medianas o grandes, donde se necesite organizar mejor el código mediante esquemas, modelos y validaciones.

Conclusión

En este trabajo práctico se pudo comprobar cómo una aplicación Node.js puede conectarse a MongoDB utilizando dos enfoques distintos.

Por un lado, el driver nativo permitió realizar una conexión directa con la base de datos y ejecutar una inserción de varios documentos mediante `insertMany()`.

Por otro lado, Mongoose permitió trabajar con un esquema definido, un modelo y una inserción de prueba, agregando mayor organización y validación dentro del código.

Ambos enfoques permiten comunicar una aplicación con MongoDB, pero se utilizan en contextos diferentes. El driver nativo ofrece mayor control y rendimiento, mientras que Mongoose brinda más estructura y facilidad de mantenimiento para aplicaciones más grandes.